

# MRI Foreground/Background Detection

Amir R. Sadri  
ars329@case.edu  
December 9, 2025

## Abstract

This note outlines how we derive foreground and background masks for MRI volumes in RadQy. Each section documents one detection method, its assumptions, and the clean-up steps needed so FG/BG pixels are reproducible across datasets.

## Contents

|     |                                                     |   |
|-----|-----------------------------------------------------|---|
| 1   | Introduction                                        | 2 |
| 2   | Image Processing Segmenters                         | 2 |
| 2.1 | OtsuHull (OtsuHull.py)                              | 2 |
| 2.2 | Adaptive Border Segmentation (AdaptiveBorderSeg.py) | 2 |
| 2.3 | Hybrid Otsu + Convex Hull (HybridOtsuHullSeg.py)    | 3 |
| 3   | Deep Learning Segmenters                            | 3 |
| 3.1 | U-Net (placeholder)                                 | 3 |
| 4   | Notes for Integration                               | 3 |
|     | References                                          | 3 |

# 1 Introduction

This document lists the foreground/background (FG/BG) segmentation methods in `backend/seg/mri`. Each section mirrors a Python module and notes its inputs, thresholds, morphology, and fallbacks so downstream IQMs get consistent masks.

## 2 Image Processing Segmenters

Classical pipelines based on intensity statistics and morphology.

### 2.1 OtsuHull (`OtsuHull.py`)

Legacy variant mirroring the original hybrid Otsu pipeline with more defensive fallbacks [1]. This subsection is listed first because it matches the earliest implementation shipped with RadQy.

**Algorithm 1:** OtsuHull foreground/background estimation

---

1. Input slice  $I$ ; cast to uint16.
2. Histogram-equalize  $I$  to get  $H$ .
3. Otsu on  $I$  and  $H$  to obtain masks  $m_1, m_2$  and FG fractions  $w_1, w_2$ .
4. Hybrid recombination:  $S = w_1 \cdot I + w_2 \cdot H$  (normalize to image range).
5. Otsu on  $S$  to get final mask  $M$ .
6. Cleanup: convex hull of  $M$  (fallback to hole filling if hull fails).
7. Outputs:  $fg = M$ ,  $bg = 1 - M$ .

Notes on the weights and masks:  $m_1$  is the Otsu mask on the raw slice  $I$ ,  $m_2$  is the Otsu mask on the hist-equalized slice  $H$ . The weights  $w_1 = |m_1|/|I|$  and  $w_2 = |m_2|/|I|$  measure how much of each slice is classified as foreground; they scale the raw and equalized images when forming  $S = w_1 \cdot I + w_2 \cdot H$  so that the cleaner mask (smaller foreground fraction) influences the recombined image more.

### 2.2 Adaptive Border Segmentation (`AdaptiveBorderSeg.py`)

Single-slice segmenter that estimates background from the image border and keeps the largest non-background component.

**Algorithm 2:** AdaptiveBorder foreground/background estimation

---

1. Normalize input slice to uint8 via 1st/99th percentile scaling.
2. Extract border ring (default 12 px each side); compute  $\mu, \sigma$  on non-zero border pixels (fallback: full image if border empty).
3. Mark background where  $I < \mu - 1\sigma$ .
4. Morphology: binary closing then opening with  $5 \times 5$  kernel; fill holes.
5. Foreground = inverse of background; keep largest connected component.
6. Safety: if FG area  $< 5\%$ , set FG to all-ones (entire slice).

7. Outputs:  $(fg, bg)$  with  $bg = 1 - fg$ .

## 2.3 Hybrid Otsu + Convex Hull (HybridOtsuHullSeg.py)

Modern hybrid approach combining Otsu on raw and histogram-equalized images with convex hull cleanup.

---

### Algorithm 3: Hybrid Otsu + Convex Hull

1. Robust-scale input slice to  $[0, 1]$  via 1st/99th percentiles; cast to uint8 for thresholding.
2. Compute Otsu masks on raw ( $x$ ) and hist-equalized ( $h$ ) images to get FG fractions  $w_1, w_2$ .
3. Recombine:  $S = w_1 \cdot x + w_2 \cdot h$ .
4. Otsu on  $S$  to obtain mask  $M$ .
5. Cleanup: apply convex hull to  $M$  (fallback: hole filling if hull unavailable).
6. Outputs:  $(fg, bg)$  with  $fg = M$ ,  $bg = 1 - fg$ .

## 3 Deep Learning Segmenters

Learned models for FG/BG inference.

### 3.1 U-Net (placeholder)

Planned slot for the learned U-Net FG/BG mask generator.

---

### Algorithm 4: U-Net inference pipeline (placeholder)

1. Preprocess: normalize/standardize input slice to network expectations.
2. Forward pass through U-Net to obtain probability map  $P$ .
3. Threshold  $P$  (e.g., at 0.5) to get binary mask  $M$ .
4. Post-process: optional hole filling, small-component removal, and convex hull if needed.
5. Outputs:  $(fg, bg)$  with  $fg = M$ ,  $bg = 1 - fg$ .

## 4 Notes for Integration

All segmenters operate per 2D slice and return uint8 masks in  $\{0, 1\}$ . Background is always defined as the complement of foreground to ensure downstream IQMs receive matching shapes and disjoint FG/BG sets.

## References

- [1] Amir Reza Sadri et al. “MRQy: An open-source tool for the quality control of structural MRI images”. In: *Journal of Imaging* 6.1 (2020). Placeholder entry; replace with canonical citation if available, pp. 1–15.